

機器學習入門

前兩週 | 用 PyTorch 從線性回歸到 MLP

家教教學投影片 · 每週 2 小時 · 理論與實作各半

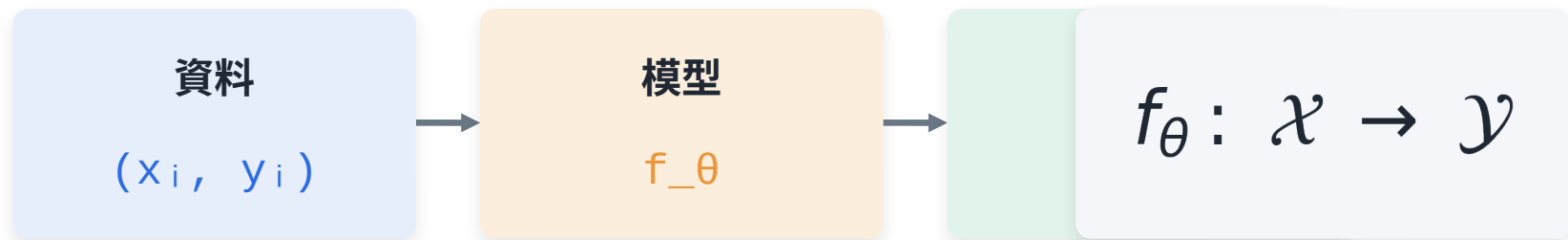


00

課程導論

Introduction — 機器學習在學什麼、用什麼數學描述

● 什麼是機器學習？



- 不是人寫死規則，而是從成對的資料 (x, y) 把規則學出來
- 監督式學習：每筆訓練資料都帶有正確答案 y
- 學到的 f 要能對「沒看過的資料」也預測得準（泛化）

● 監督式學習的數學框架

$$\theta^* = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n L(f_{\theta}(x_i), y_i)$$

找一組參數 θ , 讓「所有資料的平均損失」最小——這就是訓練。之後每個模型都是在解這道題, 只是換掉 f 和 L 。

符號	意義
x	輸入 / 特徵
y	真實標籤 (正確答案)
$\hat{y}$	模型的預測
θ (w, b)	模型參數
L	損失函數
η	學習率

● 兩週地圖：一個訓練迴圈，學會 MLP

第一週

線性回歸

建立「訓練迴圈」模板



第二週

MLP

隱藏層 + 非線性激活

共用同一個訓練迴圈：forward → loss → backward → step

第二週只換掉「模型定義」那一行，迴圈完全不變——這就是兩週能學會用 MLP 的關鍵。

01

第一週

用 PyTorch 學線性回歸，建立訓練迴圈模板

● 所有模型的共同骨架

①

模型 (model)

$$\hat{y} = w \cdot x + b$$

決定預測長什麼樣

②

損失 (loss)

MSE

量化「預測差多少」

③

最佳化 (optimization)

梯度下降

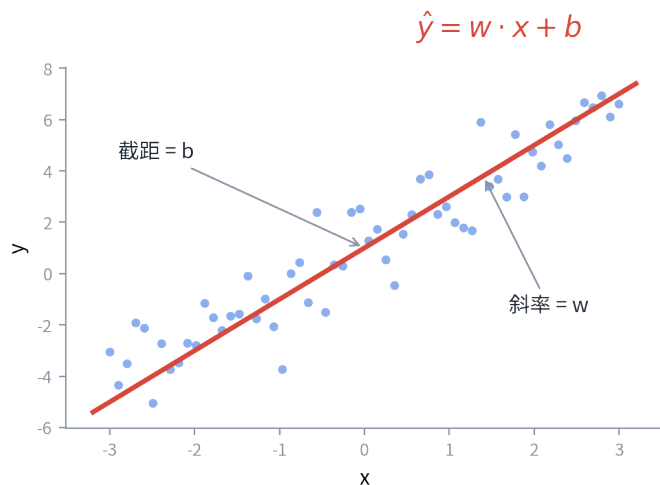
調參數讓損失變小

接下來三頁，分別把這三件事寫成數學式。

● ① 模型：線性回歸

$$\hat{y} = wx + b$$

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

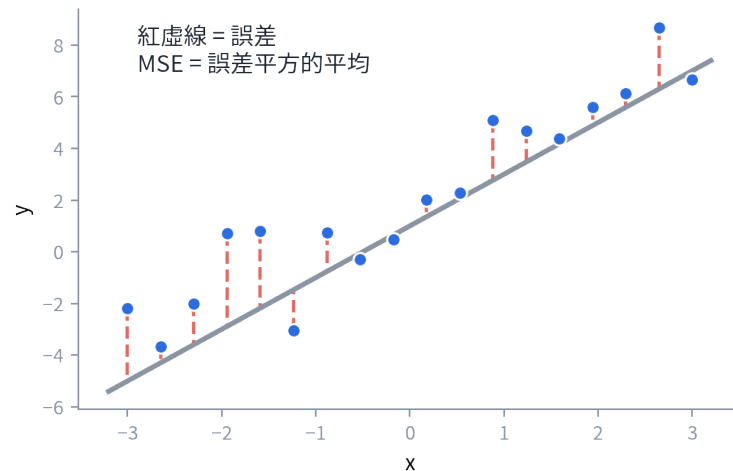


- w 控制斜率, b 控制高度
- 多特徵時: $\hat{y} = \mathbf{w}^T \mathbf{x} + b$ (向量內積)
- 訓練 = 找最貼合資料的 w, b
- 假資料真實值 $w=2$ 、 $b=1$, 可驗證學對沒

● ② 損失：均方誤差 (MSE)

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

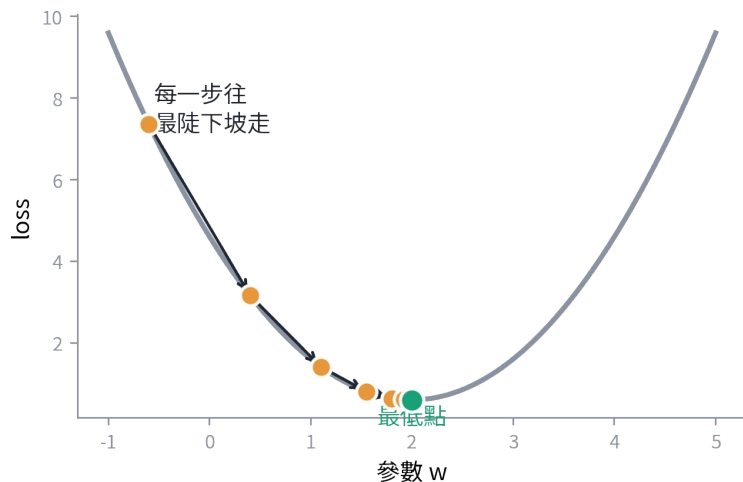
- $\hat{y}_i - y_i$ 是每個點的誤差
- 平方：大錯重罰，且處處可微分
- 對所有樣本取平均 → 一個總分
- 損失越小，線越貼合資料



● ③ 最佳化：梯度下降

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

η = 學習率 (步伐大小)



- ∇L 指向損失上升最快的方向
- 往反方向走一小步，逐步逼近谷底
- η 太大會跨過頭、發散
- η 太小則收斂很慢

● PyTorch:tensor、autograd 與訓練迴圈

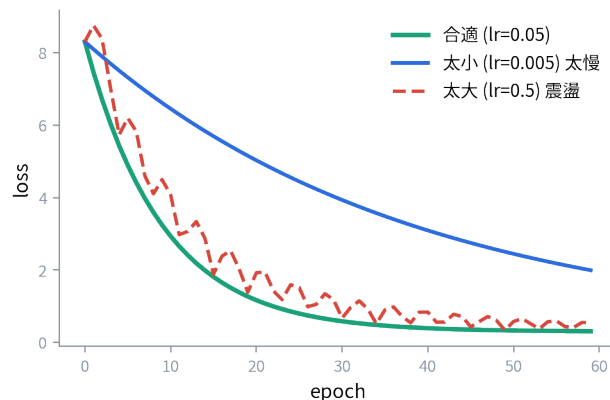
- tensor: 會記錄運算的 NumPy 陣列
- autograd: 呼叫 `.backward()` 自動算梯度
- —— 也就是公式裡的 ∇L , 不用自己推
- 右邊四步, 是接下來每堂課的模板



● 實作①：純手動梯度下降

```
w = torch.tensor(0.0)
b = torch.tensor(0.0)
lr = 0.05

for epoch in range(200):
    y_pred = w * X + b                # forward
    loss = ((y_pred - y) ** 2).mean() # MSE
    grad_w = (2*(y_pred - y) * X).mean() # grad
    grad_b = (2*(y_pred - y)).mean()
    w -= lr * grad_w                  # update
    b -= lr * grad_b
```



現場 demo: 把 lr 改成 0.5 / 0.001, 看損失曲線一路下降 vs 震盪。這段最有價值, 別省。

● 梯度為什麼長這樣 ?(原理推導)

連鎖律：對平方微分掉下 2 與括號，再乘「內部對該參數的微分」

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w}$$

對 w

$$\frac{\partial \hat{y}}{\partial w} = x \quad \frac{\partial L}{\partial w} = \frac{1}{n} \sum 2(\hat{y}_i - y_i) x_i$$

對 b

$$\frac{\partial \hat{y}}{\partial b} = 1 \quad \frac{\partial L}{\partial b} = \frac{1}{n} \sum 2(\hat{y}_i - y_i)$$

關鍵：grad_w 多乘一個 x(來自 $\partial \hat{y} / \partial w$); grad_b 沒乘，因為 $\partial \hat{y} / \partial b = 1$ 。程式的 .mean() 就是公式的平均。

● 實作② : 改用 autograd

```
w = torch.tensor(0.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)

for epoch in range(200):
    y_pred = w * X + b
    loss = ((y_pred - y) ** 2).mean()
    loss.backward()           # auto-diff grads
    with torch.no_grad():
        w -= lr * w.grad
        b -= lr * b.grad
        w.grad.zero_()       # must zero grads
        b.grad.zero_()
```

- 把「手算梯度」換成 `loss.backward()`
- 結果一樣, 但不用自己推導公式
- `requires_grad=True`: 標記要追蹤的參數
- 常見坑: 每輪梯度要歸零, 否則累加

● 實作③ :nn.Linear + optimizer = 通用模板

```
model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)

for epoch in range(200):
    y_pred = model(x)           # forward
    loss = loss_fn(y_pred, y)   # loss
    optimizer.zero_grad()       # zero grads
    loss.backward()             # backward
    optimizer.step()            # update
```

這就是模板

之後每個模型都用這個迴圈。

下週的 MLP, 只換掉
`model = ...`

那一行而已。

● 第一週作業

1 調學習率

把 lr 設成 0.5、0.01、0.001, 各畫一張損失曲線並寫下觀察

2 多特徵

資料改成兩個特徵, 用 `nn.Linear(2, 1)` 重跑一次

3 思考題

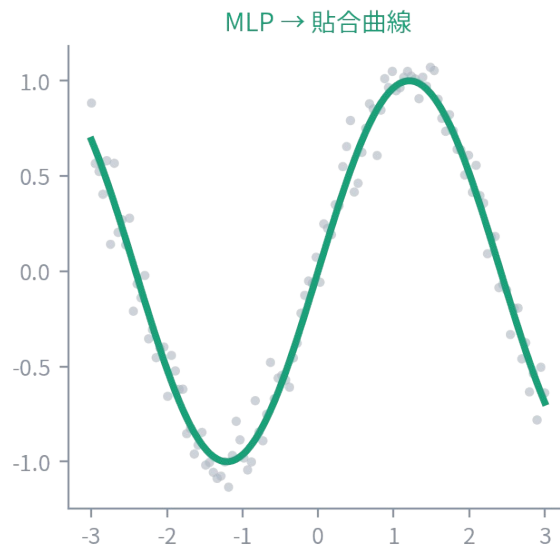
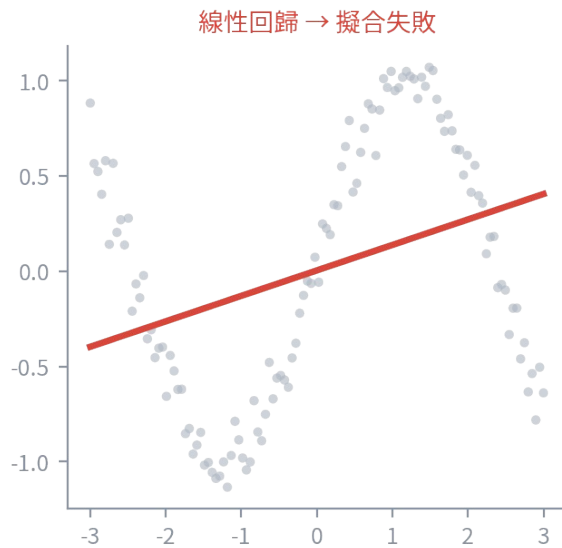
若真實關係是 $y = x^2$, 直線模型擬合得了嗎? 為什麼?

02

第二週

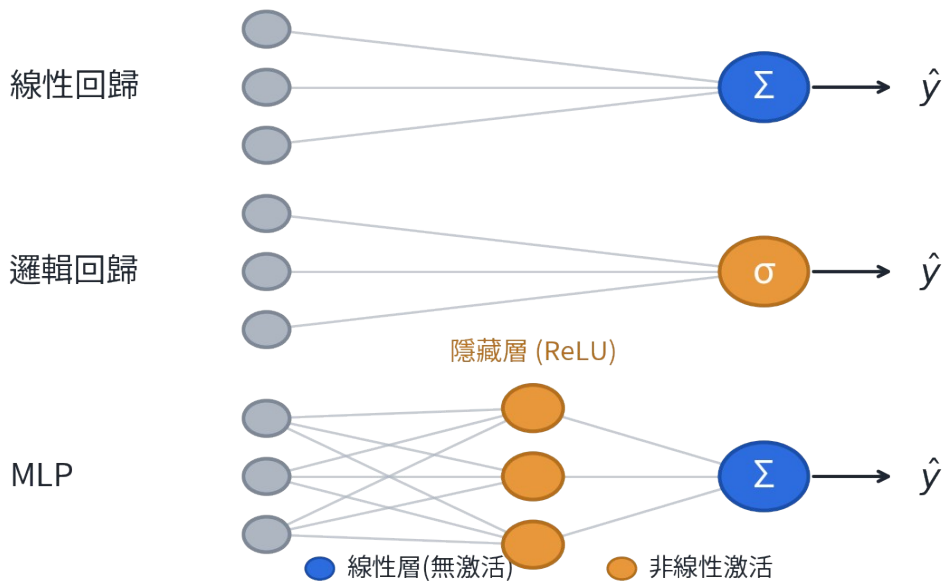
從線性回歸長成 MLP

● 先看見問題：線性模型的極限



線性模型只能表示一條直線 $\hat{y} = wx + b$, 怎麼訓練都貼不上正弦波。問學員：缺了什麼？(答案：非線性。)

● MLP 核心：隱藏層 + 非線性激活



$$\mathbf{h} = \text{ReLU}(W_1\mathbf{x} + \mathbf{b}_1)$$

$$\hat{y} = W_2\mathbf{h} + \mathbf{b}_2$$

- W_1 、 W_2 是各層的權重矩陣
- ReLU 提供非線性，把直線折彎
- 線性回歸 = 最簡單的單層網路

● 為什麼一定要激活函數？

假設沒有激活函數，把兩層線性層直接接起來：

$$W_2(W_1\mathbf{x}) = (W_2W_1)\mathbf{x} = W'\mathbf{x}$$

兩層線性 = 一層線性。中間沒有非線性，整個模型就「塌回」線性回歸——疊再多層都白疊。

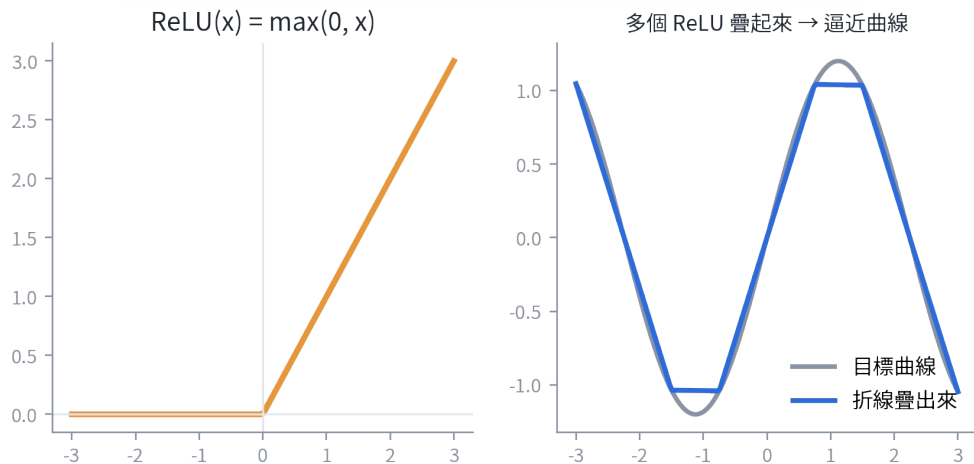
加入 ReLU 這個非線性，多層才真正有意義

$$\text{ReLU}(z) = \max(0, z)$$

反向傳播 = 上週的 `.backward()`, PyTorch 自動算多層梯度，不是新東西。

● ReLU 把直線折彎，逼近任意曲線

$$\text{ReLU}(z) = \max(0, z)$$



單個 ReLU 只是一個轉折；很多個 ReLU 疊加，就能拼出任意彎曲的曲線——這就是 MLP 表達能力的來源。

● 實作①：用 MLP 擬合曲線

```
mlp = nn.Sequential(
    nn.Linear(1, 32),
    nn.ReLU(),          # <-- the nonlinearity
    nn.Linear(32, 1),
)
optimizer = torch.optim.Adam(mlp.parameters(), lr=0.01)

for epoch in range(1000):    # same loop as week 1
    loss = loss_fn(mlp(X), y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

- 訓練迴圈跟上週一模一樣
- 只換掉「模型定義」那幾行
- 玩玩看：神經元 $32 \rightarrow 4 \rightarrow 128$ 的差別
- 刪掉 ReLU $\rightarrow$ 模型塌回直線 (現場 demo)

● 分類的數學 : Softmax + 交叉熵

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

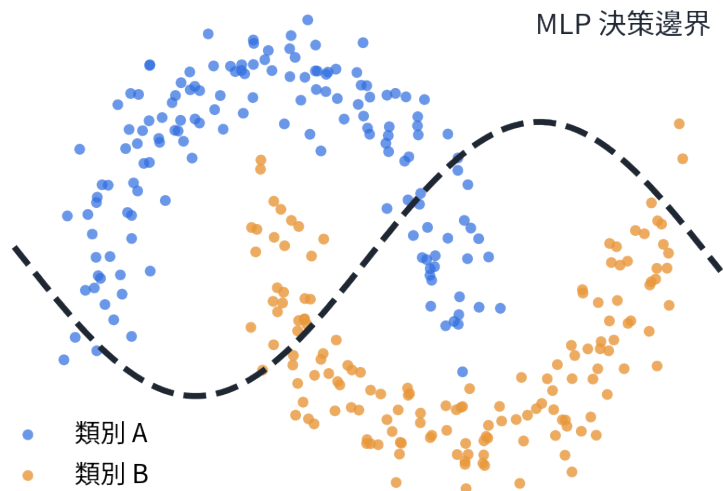
Softmax 把模型輸出轉成各類別的機率，總和為 1。

$$L = -\sum_k y_k \log \hat{y}_k$$

交叉熵衡量「預測機率」與「真實類別」的差距——它就是分類版的 MSE。

PyTorch 小提醒 : `nn.CrossEntropyLoss` 已內含 softmax, 所以模型最後一層直接輸出原始分數 (logits) 即可。

● 實作②：用 MLP 做分類



```
clf = nn.Sequential(  
    nn.Linear(2, 16), nn.ReLU(),  
    nn.Linear(16, 2),      # 2 classes  
)  
loss_fn = nn.CrossEntropyLoss()  
opt = torch.optim.Adam(clf.parameters(), lr=0.01)  
  
for epoch in range(500):    # same loop again  
    loss = loss_fn(clf(X), y)  
    opt.zero_grad(); loss.backward(); opt.step()  
  
acc = (clf(X).argmax(1) == y).float().mean()
```

重點不是 *cross-entropy* 的數學細節，而是：換個資料和損失函數，同一個模板就能做分類。

兩週總結

訓練迴圈是模板

你已經能蓋 MLP、訓練它，並用在回歸與分類兩種任務。

而核心，就是那個重複了無數次的迴圈：

forward → loss → backward → step

下一步：模型評估、過擬合、怎麼判斷模型學得好不好。